

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**Nhóm 8
Nhóm trưởng: Nguyễn Đình Vinh Lộc
Thành viên: Bùi Thái Sơn
Nguyễn Phú Lâm
Trần Thị Phương Thảo
Lê Hoài Nam
Phạm Quang Nhật**

DATABASE TRONG KIẾN TRÚC MICROSERVICES

**BÁO CÁO BÀI TẬP
Ngành: Hệ thống thông tin**

HÀ NỘI – 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**

**Nhóm 8
Nhóm trưởng: Nguyễn Đình Vinh Lộc
Thành viên: Bùi Thái Sơn
Nguyễn Phú Lâm
Trần Thị Phương Thảo
Lê Hoài Nam
Phạm Quang Nhật**

DATABASE TRONG KIẾN TRÚC MICROSERVICES

**BÁO CÁO BÀI TẬP
Ngành: Hệ thống thông tin**

Cán bộ hướng dẫn: TS. Vương Thị Hồng

HÀ NỘI – 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**

Team 8

Leader: Nguyen Dinh Vinh Loc

Members: Bui Thai Son

Nguyen Phu Lam

Tran Thi Phuong Thao

Le Hoai Nam

Pham Quang Nhat

DATABASE IN MICROSERVICES ARCHITECTURE

PROJECT REPORT

Major: Information system

Supervisor: Assoc. Prof., M.s.Vương Thị Hồng

HÀ NỘI – 2026

Mục lục

Chương 1. Mở đầu	1
1.1. Bối cảnh và lý do chọn đề tài	1
1.2. Vấn đề cần giải quyết	2
1.3. Mục tiêu của đề tài	3
1.4. Phạm vi và giới hạn	3
1.5. Phương pháp thực hiện	3
1.6. Cấu trúc báo cáo	4
Chương 2. Cơ sở lý thuyết	5
2.1. Kiến trúc Microservices	5
2.2. Lựa chọn cơ sở dữ liệu phù hợp	5
2.2.1. Relational Database	5
2.2.2. Document Database	6
2.2.3. Key-Value Database	6
2.2.4. Search Engine Database	6
2.2.5. Graph Database	6
2.3. Giao tiếp giữa các service	7
2.3.1. REST API	7
2.3.2. gRPC	7
2.3.3. Message Queue	7
2.4. Thách thức về tính nhất quán dữ liệu	8
2.4.1. CAP Theorem	8
2.4.2. Các thách thức phổ biến	8
2.5. Các mô hình quản lý cơ sở dữ liệu trong Microservices	8
2.5.1. Shared Database / Shared Schema	8
2.5.2. Database per Service	9
2.5.3. Schema per Service	9
2.6. Các Design Pattern phổ biến	10
2.6.1. CQRS Pattern	10

2.6.2. Saga Pattern.....	10
2.7. Khi nào không nên sử dụng Microservices.....	13
2.8. Triển khai và cấu hình.....	14
Chương 3. Phân tích và Thiết kế hệ thống.....	15
3.1. Yêu cầu chức năng.....	15
3.2. Kiến trúc tổng thể	15
3.3. Thiết kế cơ sở dữ liệu	16
3.4. Thiết kế API	18
3.5. Luồng nghiệp vụ chính	18
Chương 4. Thực nghiệm và Kết quả	19
4.1. Môi trường thực nghiệm.....	19
4.2. Dữ liệu mẫu	19
4.3. Kịch bản kiểm thử.....	19
4.3.1. Check từng service	20
4.3.2. Tạo người dùng và sản phẩm.....	22
4.3.3. Tạo đơn hàng với nhiều sản phẩm	23
4.3.4. Lấy danh sách đơn hàng.....	23
4.4. Kết quả đạt được	24
Chương 5. Kết luận và Hướng phát triển	25
5.1. Kết luận	25
5.2. Hạn chế.....	25
5.3. Hướng phát triển.....	25

Danh sách hình vẽ

2.2 Database per Service	9
2.2.	9
2.4 CQRS Pattern	10
2.4.	10
2.6 Saga Pattern.....	11
2.6.	11
2.8 Choreography	12
2.8.	12
2.10 Orchestration	13
2.10.	13
3.2 Kiến trúc hệ thống.....	15
3.2.	15
3.4 Sơ đồ EER của user service.....	16
3.4.	16
3.6 Sơ đồ EER của product service.....	17
3.6.	17
3.8 Sơ đồ EER của order service	17
3.8.	17
4.2 Check user service.....	20
4.2.	20
4.4 Check product service.....	21
4.4.	21
4.6 Check order service	21
4.6.	21
4.8 Tạo người dùng	22
4.8.	22
4.10 Tạo sản phẩm.....	22
4.10.	22

4.12 Tạo đơn hàng với nhiều sản phẩm.....	23
4.12.	23
4.14 Lấy danh sách đơn hàng	23
4.14.	23
4.16 Kết quả 12 test case CRUD cho User Service, Product Service và Order Service	24
4.16.	24

Chương 1.

Mở đầu

1.1. Bối cảnh và lý do chọn đề tài

Trong thời đại chuyển đổi số, các hệ thống phần mềm ngày càng có quy mô lớn, phục vụ số lượng người dùng ngày càng tăng và yêu cầu khả năng mở rộng linh hoạt. Những ứng dụng hiện đại như thương mại điện tử, dịch vụ xem phim trực tuyến, gọi xe công nghệ hay mạng xã hội phải xử lý khối lượng dữ liệu khổng lồ cùng hàng triệu yêu cầu truy cập mỗi ngày. Điều này đặt ra yêu cầu cao về hiệu năng, tính sẵn sàng và khả năng bảo trì của hệ thống.

Trước đây, kiến trúc Monolithic được sử dụng phổ biến do tính đơn giản trong thiết kế và triển khai. Tuy nhiên, khi hệ thống phát triển lớn hơn, kiến trúc này bộc lộ nhiều hạn chế như khó mở rộng, khó bảo trì, thời gian triển khai kéo dài và sự phụ thuộc chặt chẽ giữa các thành phần. Để khắc phục những hạn chế đó, kiến trúc Microservices đã ra đời và nhanh chóng trở thành xu hướng trong phát triển phần mềm hiện đại.

Microservices cho phép chia ứng dụng thành nhiều dịch vụ nhỏ, độc lập và có thể triển khai riêng biệt. Mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ cụ thể và có thể sử dụng công nghệ phù hợp với yêu cầu của mình. Tuy nhiên, việc phân tách hệ thống thành nhiều dịch vụ cũng làm phát sinh các vấn đề mới liên quan đến quản lý dữ liệu, giao tiếp giữa các dịch vụ và đảm bảo tính nhất quán dữ liệu trong môi trường phân tán.

Đặc biệt, việc lựa chọn mô hình cơ sở dữ liệu phù hợp cho từng dịch vụ, xây dựng cơ chế đồng bộ dữ liệu và áp dụng các mẫu thiết kế như CQRS hay Saga Pattern đóng vai trò quan trọng trong sự thành công của hệ thống Microservices. Đây là những vấn đề mang tính thực tiễn cao và được nhiều doanh nghiệp công nghệ lớn như Netflix hay Shopee áp dụng trong quá trình phát triển hệ thống.

Xuất phát từ những lý do trên, nhóm lựa chọn đề tài "Database in Microservices" nhằm tìm hiểu các mô hình quản lý cơ sở dữ liệu trong kiến trúc Microservices, các thách thức về tính nhất quán dữ liệu cũng như các giải pháp và mẫu thiết kế phổ biến được sử dụng để giải quyết những vấn đề này. Thông qua đó, nhóm mong muốn có cái nhìn tổng quan về việc tổ chức và quản lý dữ liệu trong các hệ thống phần mềm hiện đại, đồng thời nâng cao kiến thức và kỹ năng thiết kế hệ thống phân tán trong thực tế.

1.2. Vấn đề cần giải quyết

Khi triển khai kiến trúc Microservices, mỗi dịch vụ được tách biệt và hoạt động độc lập nhằm tăng khả năng mở rộng, bảo trì và triển khai. Tuy nhiên, việc phân tách hệ thống thành nhiều dịch vụ nhỏ cũng làm phát sinh nhiều vấn đề liên quan đến quản lý dữ liệu mà kiến trúc Monolithic ít gặp phải.

Trước hết, mỗi microservice thường sở hữu cơ sở dữ liệu riêng để đảm bảo tính độc lập. Điều này khiến việc chia sẻ và đồng bộ dữ liệu giữa các dịch vụ trở nên khó khăn hơn. Khi một nghiệp vụ liên quan đến nhiều dịch vụ khác nhau, dữ liệu có thể không được cập nhật đồng thời, dẫn đến tình trạng mất tính nhất quán.

Bên cạnh đó, các giao dịch phân tán không thể dễ dàng sử dụng cơ chế ACID Transaction như trong hệ thống sử dụng một cơ sở dữ liệu tập trung. Nếu một bước trong quy trình xử lý thất bại, hệ thống cần có cơ chế khôi phục hoặc hoàn tác phù hợp để tránh dữ liệu bị sai lệch giữa các dịch vụ.

Ngoài ra, việc truy vấn dữ liệu giữa nhiều microservice cũng gặp nhiều thách thức. Các dịch vụ không được phép truy cập trực tiếp cơ sở dữ liệu của nhau nhằm đảm bảo tính đóng gói và giảm phụ thuộc. Do đó, các thao tác tổng hợp dữ liệu phải được thực hiện thông qua API hoặc cơ chế trao đổi sự kiện, làm tăng độ phức tạp của hệ thống.

Một vấn đề khác là lựa chọn mô hình cơ sở dữ liệu phù hợp cho từng dịch vụ. Mỗi loại dữ liệu có những đặc điểm khác nhau nên việc sử dụng một loại cơ sở dữ liệu duy nhất cho toàn bộ hệ thống có thể không đạt hiệu quả tối ưu. Điều này đòi hỏi phải nghiên cứu các mô hình như Shared Database, Database per Service hay Schema per Service để lựa chọn giải pháp phù hợp.

Từ những khó khăn trên, đề tài tập trung giải quyết các vấn đề chính sau:

- Tìm hiểu các mô hình tổ chức cơ sở dữ liệu trong kiến trúc Microservices.
- Phân tích ưu điểm, nhược điểm của từng mô hình quản lý dữ liệu.
- Nghiên cứu các thách thức về tính nhất quán dữ liệu trong hệ thống phân tán.
- Tìm hiểu các phương thức giao tiếp dữ liệu giữa các microservices.
- Nghiên cứu các mẫu thiết kế phổ biến như CQRS và Saga Pattern nhằm xử lý giao dịch phân tán và đảm bảo tính nhất quán dữ liệu.
- Đề xuất mô hình quản lý dữ liệu phù hợp cho các hệ thống Microservices hiện đại.

Thông qua việc nghiên cứu các nội dung trên, đề tài hướng đến mục tiêu làm rõ cách thức quản lý cơ sở dữ liệu hiệu quả trong kiến trúc Microservices, từ đó

giúp xây dựng các hệ thống có khả năng mở rộng tốt, dễ bảo trì và đảm bảo tính ổn định trong môi trường phân tán.

1.3. Mục tiêu của đề tài

Đề tài được thực hiện nhằm nghiên cứu cách tổ chức và quản lý cơ sở dữ liệu trong kiến trúc Microservices, từ đó tìm hiểu các mô hình lưu trữ dữ liệu, phương thức giao tiếp giữa các dịch vụ và những thách thức liên quan đến tính nhất quán dữ liệu trong hệ thống phân tán.

1.4. Phạm vi và giới hạn

Đề tài tập trung nghiên cứu việc quản lý cơ sở dữ liệu trong kiến trúc Microservices thông qua mô hình hệ thống bán hàng đơn giản gồm ba dịch vụ chính: User Service, Product Service và Order Service. Mỗi dịch vụ được thiết kế theo nguyên tắc sở hữu cơ sở dữ liệu riêng nhằm minh họa mô hình Database per Service và cách tổ chức dữ liệu trong môi trường Microservices.

Do giới hạn về thời gian và phạm vi nghiên cứu, đề tài chỉ tập trung vào việc xây dựng và mô phỏng các chức năng cơ bản của hệ thống, chưa triển khai các cơ chế đồng bộ dữ liệu phức tạp giữa nhiều cơ sở dữ liệu. Ngoài ra, các thành phần nâng cao thường được sử dụng trong hệ thống Microservices thực tế như Message Broker (Kafka, RabbitMQ), Saga Pattern, Distributed Tracing, API Gateway hay các giải pháp giám sát và quản lý tập trung cũng chưa được tích hợp trong mô hình minh họa. Vì vậy, kết quả của đề tài chủ yếu mang tính nghiên cứu và mô phỏng các nguyên tắc cơ bản về quản lý dữ liệu trong kiến trúc Microservices.

1.5. Phương pháp thực hiện

Đề tài được thực hiện thông qua việc nghiên cứu các tài liệu về kiến trúc Microservices, đặc biệt là mô hình Database per Service và các phương pháp quản lý dữ liệu trong hệ thống phân tán. Trên cơ sở lý thuyết đã khảo sát, nhóm tiến hành phân tích yêu cầu, thiết kế kiến trúc hệ thống và xây dựng cơ sở dữ liệu riêng cho từng dịch vụ gồm User Service, Product Service và Order Service. Hệ thống được triển khai bằng NodeJS kết hợp Express để xây dựng các API, sử dụng MySQL làm hệ quản trị cơ sở dữ liệu. Sau khi hoàn thành việc cài đặt, nhóm chuẩn bị dữ liệu mẫu và thực hiện kiểm thử các chức năng thông qua Postman nhằm đánh giá khả năng hoạt động và giao tiếp giữa các dịch vụ trong mô hình Microservices.

1.6. Cấu trúc báo cáo

- Chương 1 trình bày tổng quan về đề tài, bao gồm bối cảnh, lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi thực hiện và phương pháp nghiên cứu.
- Chương 2 trình bày cơ sở lý thuyết về kiến trúc Microservices, các mô hình cơ sở dữ liệu, phương thức giao tiếp giữa các dịch vụ và những thách thức về tính nhất quán dữ liệu.
- Chương 3 trình bày quá trình phân tích, thiết kế hệ thống và thiết kế cơ sở dữ liệu cho các dịch vụ trong mô hình Microservices.
- Chương 4 mô tả việc xây dựng hệ thống, triển khai thực nghiệm, kiểm thử API và đánh giá kết quả đạt được.
- Chương 5 trình bày kết luận, những kết quả đạt được của đề tài và đề xuất hướng phát triển trong tương lai.

Chương 2.

Cơ sở lý thuyết

2.1. Kiến trúc Microservices

Microservice là một phong cách kiến trúc phần mềm trong đó ứng dụng được chia thành nhiều dịch vụ nhỏ, độc lập và tự quản lý. Mỗi dịch vụ tập trung thực hiện một chức năng nghiệp vụ cụ thể, có thể được phát triển, triển khai và mở rộng riêng biệt mà không ảnh hưởng đến các dịch vụ khác.

Các microservice giao tiếp với nhau thông qua các giao thức mạng như REST API, gRPC hoặc Message Queue. Mỗi service thường sở hữu cơ sở dữ liệu riêng nhằm đảm bảo tính độc lập và giảm sự phụ thuộc giữa các thành phần trong hệ thống. Điều này giúp các nhóm phát triển có thể sử dụng công nghệ và hệ quản trị cơ sở dữ liệu phù hợp với từng nghiệp vụ.

Kiến trúc Microservice mang lại nhiều lợi ích như khả năng mở rộng linh hoạt, tăng tính sẵn sàng của hệ thống, dễ bảo trì và hỗ trợ triển khai liên tục. Khi một dịch vụ gặp sự cố, các dịch vụ khác vẫn có thể hoạt động bình thường, giúp giảm ảnh hưởng đến toàn bộ hệ thống.

Tuy nhiên, việc chia nhỏ hệ thống thành nhiều dịch vụ cũng làm tăng độ phức tạp trong quản lý, giám sát và đồng bộ dữ liệu. Các vấn đề như giao dịch phân tán, tính nhất quán dữ liệu và giao tiếp giữa các dịch vụ cần được giải quyết bằng các cơ chế và mẫu thiết kế phù hợp.

Kiến trúc Microservice hiện đang được áp dụng rộng rãi trong các hệ thống quy mô lớn như các nền tảng thương mại điện tử, dịch vụ phát trực tuyến, mạng xã hội và các ứng dụng điện toán đám mây nhờ khả năng đáp ứng tốt các yêu cầu về hiệu năng và khả năng mở rộng.

2.2. Lựa chọn cơ sở dữ liệu phù hợp

Trong kiến trúc Microservices, mỗi dịch vụ có thể sử dụng loại cơ sở dữ liệu phù hợp với đặc điểm nghiệp vụ và yêu cầu xử lý dữ liệu của mình. Việc lựa chọn đúng loại cơ sở dữ liệu giúp tối ưu hiệu năng, khả năng mở rộng và tính linh hoạt của hệ thống.

2.2.1. Relational Database

Relational Database là loại cơ sở dữ liệu lưu trữ dữ liệu dưới dạng các bảng có mối quan hệ với nhau. Các hệ quản trị phổ biến bao gồm MySQL và PostgreSQL.

Loại cơ sở dữ liệu này phù hợp với các hệ thống có dữ liệu được tổ chức rõ ràng, yêu cầu đảm bảo tính toàn vẹn và nhất quán dữ liệu theo nguyên tắc ACID. Ngoài ra, Relational Database cũng là lựa chọn phù hợp khi dữ liệu có nhiều mối quan hệ phức tạp cần được quản lý và truy vấn.

2.2.2. Document Database

Document Database lưu trữ dữ liệu dưới dạng tài liệu (document), thường sử dụng định dạng JSON hoặc BSON. Một số hệ quản trị phổ biến là MongoDB và CouchDB.

Loại cơ sở dữ liệu này thích hợp cho các ứng dụng có cấu trúc dữ liệu thay đổi thường xuyên, dữ liệu phi cấu trúc hoặc bán cấu trúc. Document Database thường được sử dụng trong các hệ thống quản lý nội dung, danh mục sản phẩm hoặc các ứng dụng cần tính linh hoạt cao về dữ liệu.

2.2.3. Key-Value Database

Key-Value Database lưu trữ dữ liệu theo cặp khóa – giá trị, cho phép truy xuất dữ liệu với tốc độ rất cao. Các hệ quản trị phổ biến gồm Redis và DynamoDB.

Loại cơ sở dữ liệu này thường được sử dụng cho việc lưu trữ cache, session người dùng hoặc các dữ liệu cần truy xuất nhanh nhằm cải thiện hiệu năng của hệ thống.

2.2.4. Search Engine Database

Search Engine Database được thiết kế chuyên biệt cho các chức năng tìm kiếm dữ liệu. Elasticsearch là một trong những công nghệ phổ biến nhất hiện nay.

Loại cơ sở dữ liệu này hỗ trợ tìm kiếm toàn văn (Full-text Search), cho phép truy vấn nhanh trên tập dữ liệu lớn và thường được sử dụng trong các hệ thống thương mại điện tử, quản lý tài liệu hoặc phân tích dữ liệu.

2.2.5. Graph Database

Graph Database được thiết kế để quản lý và phân tích các mối quan hệ phức tạp giữa các đối tượng dữ liệu. Các công nghệ tiêu biểu gồm Neo4j và Amazon Neptune.

Loại cơ sở dữ liệu này thường được ứng dụng trong mạng xã hội, hệ thống gợi ý sản phẩm, phân tích mạng lưới quan hệ hoặc các bài toán yêu cầu xử lý dữ liệu dạng đồ thị.

2.3. Giao tiếp giữa các service

Trong kiến trúc Microservices, các dịch vụ cần trao đổi dữ liệu với nhau để phối hợp thực hiện các chức năng nghiệp vụ. Việc giao tiếp giữa các dịch vụ thường được thực hiện thông qua API hoặc các cơ chế truyền thông điệp.

2.3.1. REST API

REST API là phương thức giao tiếp phổ biến nhất trong các hệ thống Microservices. REST sử dụng giao thức HTTP hoặc HTTPS để truyền dữ liệu giữa các dịch vụ theo mô hình yêu cầu – phản hồi (Request – Response).

Ưu điểm của REST API là đơn giản, dễ triển khai, dễ tích hợp và được hỗ trợ bởi hầu hết các nền tảng phát triển phần mềm hiện nay. Do đó, REST API phù hợp với nhiều loại ứng dụng và thường được sử dụng trong các hệ thống Microservices quy mô vừa và nhỏ.

2.3.2. gRPC

gRPC là một cơ chế gọi thủ tục từ xa (Remote Procedure Call) có hiệu năng cao do Google phát triển. gRPC sử dụng Protocol Buffers để định nghĩa và truyền dữ liệu giữa các dịch vụ.

So với REST API, gRPC có tốc độ xử lý nhanh hơn và kích thước dữ liệu truyền tải nhỏ hơn. Vì vậy, gRPC thường được sử dụng trong các hệ thống Microservices lớn hoặc các ứng dụng yêu cầu hiệu năng cao và giao tiếp nội bộ thường xuyên giữa các dịch vụ.

2.3.3. Message Queue

Message Queue là phương thức giao tiếp bất đồng bộ giữa các dịch vụ thông qua hàng đợi tin nhắn. Các công nghệ phổ biến hiện nay bao gồm Kafka và RabbitMQ.

Với phương pháp này, dịch vụ gửi thông điệp vào hàng đợi và không cần chờ phản hồi ngay lập tức. Điều này giúp giảm sự phụ thuộc giữa các dịch vụ, tăng khả năng mở rộng và hỗ trợ tốt cho kiến trúc hướng sự kiện (Event-Driven Architecture).

Một nguyên tắc quan trọng trong Microservices là các dịch vụ không được truy cập trực tiếp cơ sở dữ liệu của nhau mà phải trao đổi dữ liệu thông qua API hoặc cơ chế truyền thông điệp.

2.4. Thách thức về tính nhất quán dữ liệu

Khi mỗi microservice sở hữu cơ sở dữ liệu riêng, việc đảm bảo dữ liệu luôn nhất quán trên toàn hệ thống trở thành một thách thức lớn. Điều này xuất phát từ đặc điểm phân tán của kiến trúc Microservices.

2.4.1. CAP Theorem

Theo định lý CAP, một hệ thống phân tán chỉ có thể đảm bảo đồng thời tối đa hai trong ba thuộc tính gồm Consistency (tính nhất quán), Availability (tính sẵn sàng) và Partition Tolerance (khả năng chịu phân vùng mạng).

Trong thực tế, các hệ thống Microservices thường ưu tiên Availability và Partition Tolerance nhằm đảm bảo hệ thống luôn hoạt động ngay cả khi xảy ra sự cố mạng. Do đó, dữ liệu thường được quản lý theo mô hình Eventual Consistency, nghĩa là dữ liệu sẽ đạt trạng thái nhất quán sau một khoảng thời gian nhất định.

2.4.2. Các thách thức phổ biến

Một trong những khó khăn lớn nhất là không thể thực hiện giao dịch ACID xuyên suốt nhiều dịch vụ như trong hệ thống sử dụng một cơ sở dữ liệu tập trung. Điều này khiến việc quản lý giao dịch trở nên phức tạp hơn.

Ngoài ra, dữ liệu có thể chưa được đồng bộ ngay lập tức giữa các dịch vụ, dẫn đến trạng thái tạm thời không nhất quán. Để tăng hiệu năng, dữ liệu đôi khi được sao chép ở nhiều nơi, làm phát sinh vấn đề trùng lặp dữ liệu.

Bên cạnh đó, các dịch vụ không thể thực hiện các câu lệnh JOIN trực tiếp giữa các cơ sở dữ liệu khác nhau. Khi một dịch vụ gặp lỗi trong quá trình xử lý nghiệp vụ liên quan đến nhiều dịch vụ khác, dữ liệu của toàn hệ thống cũng có nguy cơ mất tính nhất quán.

2.5. Các mô hình quản lý cơ sở dữ liệu trong Microservices

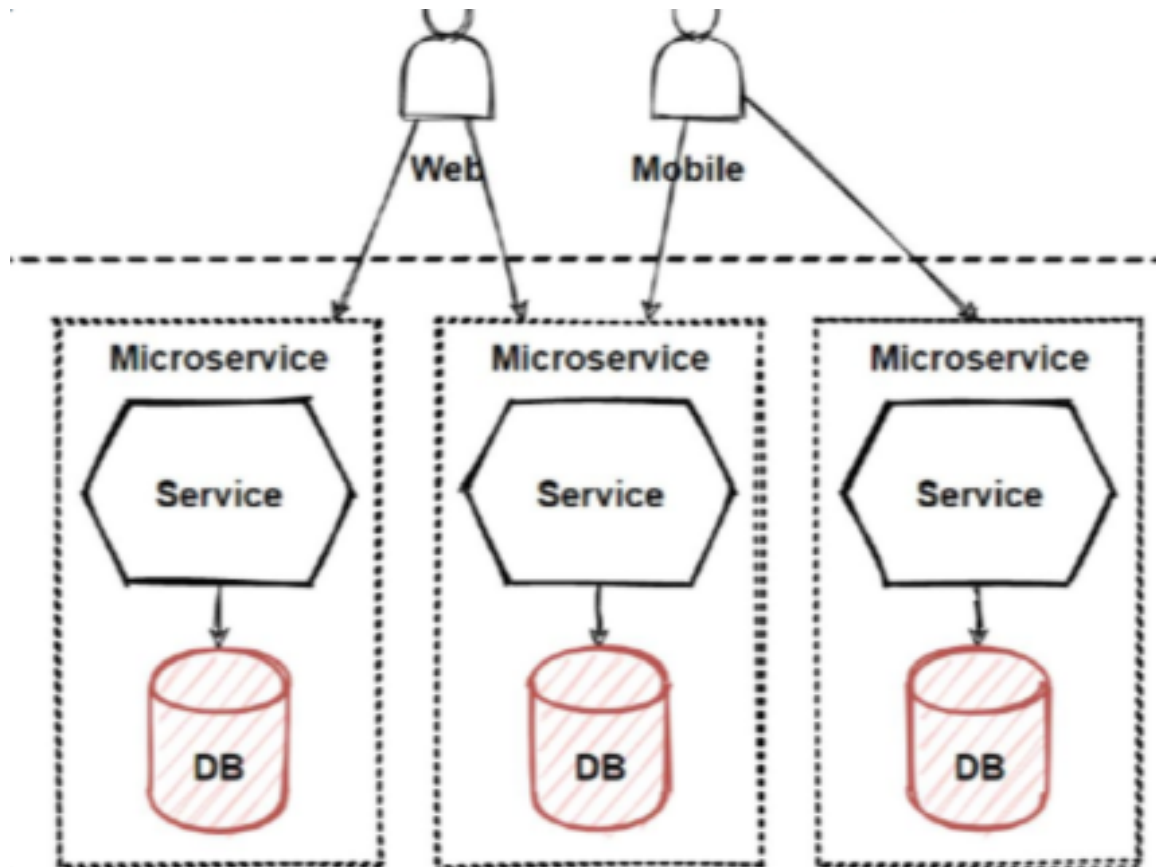
2.5.1. Shared Database / Shared Schema

Trong mô hình Shared Database, nhiều dịch vụ cùng sử dụng một cơ sở dữ liệu hoặc một schema chung.

Ưu điểm của mô hình này là dễ quản lý và giảm số lượng cơ sở dữ liệu cần vận hành. Tuy nhiên, việc nhiều dịch vụ cùng truy cập dữ liệu làm tăng sự phụ thuộc giữa các thành phần và gây khó khăn trong việc mở rộng hoặc thay đổi hệ thống.

2.5.2. Database per Service

Database per Service là mô hình trong đó mỗi dịch vụ sở hữu cơ sở dữ liệu riêng và chỉ được phép truy cập dữ liệu của chính mình.



2.2: Database per Service

Hình 2.2.

Mô hình này giúp các dịch vụ hoạt động độc lập, dễ mở rộng và cho phép lựa chọn công nghệ cơ sở dữ liệu phù hợp với từng nghiệp vụ. Đây là mô hình được sử dụng phổ biến và được xem là nguyên tắc cốt lõi của kiến trúc Microservices hiện đại.

2.5.3. Schema per Service

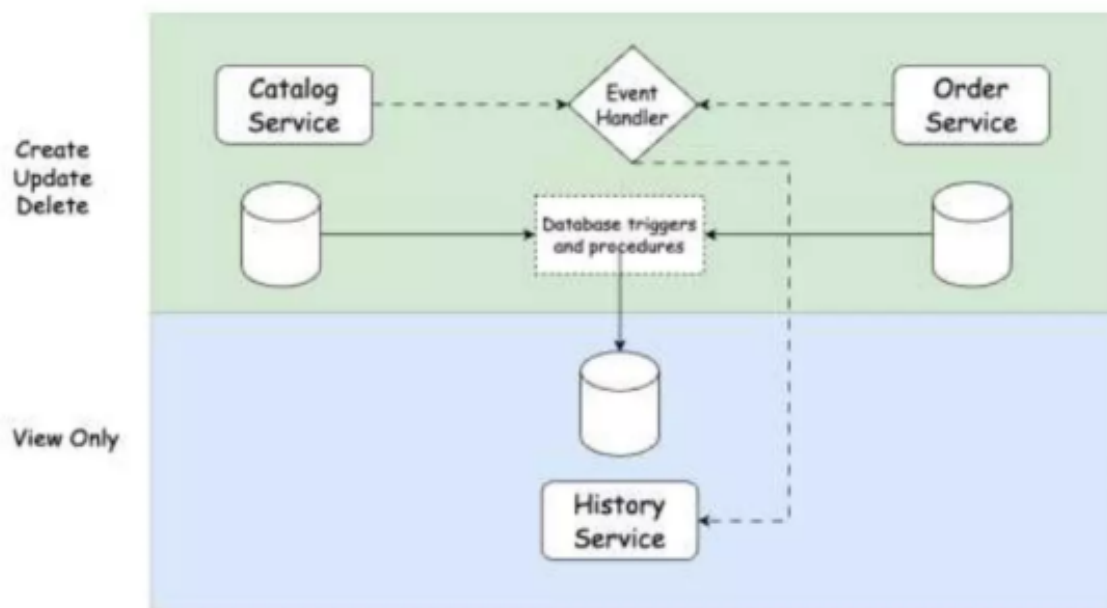
Trong mô hình Schema per Service, các dịch vụ sử dụng chung một máy chủ cơ sở dữ liệu nhưng mỗi dịch vụ có schema riêng biệt.

Giải pháp này giúp tiết kiệm tài nguyên hơn so với Database per Service, đồng thời vẫn đảm bảo mức độ cô lập dữ liệu cao hơn Shared Schema. Ngoài ra, mô hình còn hỗ trợ mở rộng và bảo trì hệ thống hiệu quả hơn.

2.6. Các Design Pattern phổ biến

2.6.1. CQRS Pattern

CQRS (Command Query Responsibility Segregation) là mẫu thiết kế tách biệt các thao tác ghi dữ liệu (Command) và đọc dữ liệu (Query) thành hai thành phần riêng biệt.



2.4: CQRS Pattern

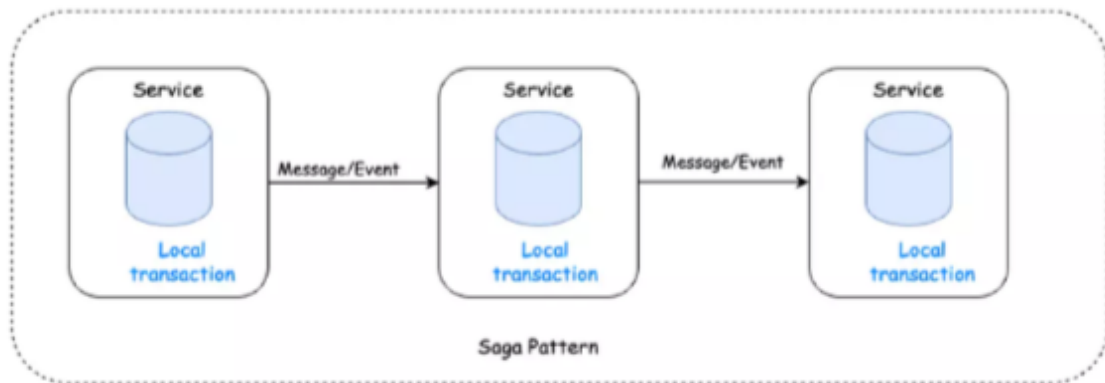
Hình 2.4.

Việc tách biệt này giúp tối ưu hiệu năng của từng thành phần, cho phép hệ thống mở rộng độc lập giữa hoạt động đọc và ghi dữ liệu. CQRS đặc biệt phù hợp với các hệ thống có lượng truy vấn lớn hoặc yêu cầu hiệu năng cao.

2.6.2. Saga Pattern

Saga Pattern là giải pháp quản lý giao dịch phân tán trong môi trường Microservices. Thay vì sử dụng giao dịch phân tán truyền thống, Saga chia quy trình nghiệp vụ thành nhiều giao dịch cục bộ tại từng dịch vụ.

Khi một bước xử lý thất bại, các giao dịch bù (Compensating Transactions) sẽ được thực hiện để hoàn tác các thay đổi trước đó nhằm đảm bảo tính nhất quán dữ liệu.

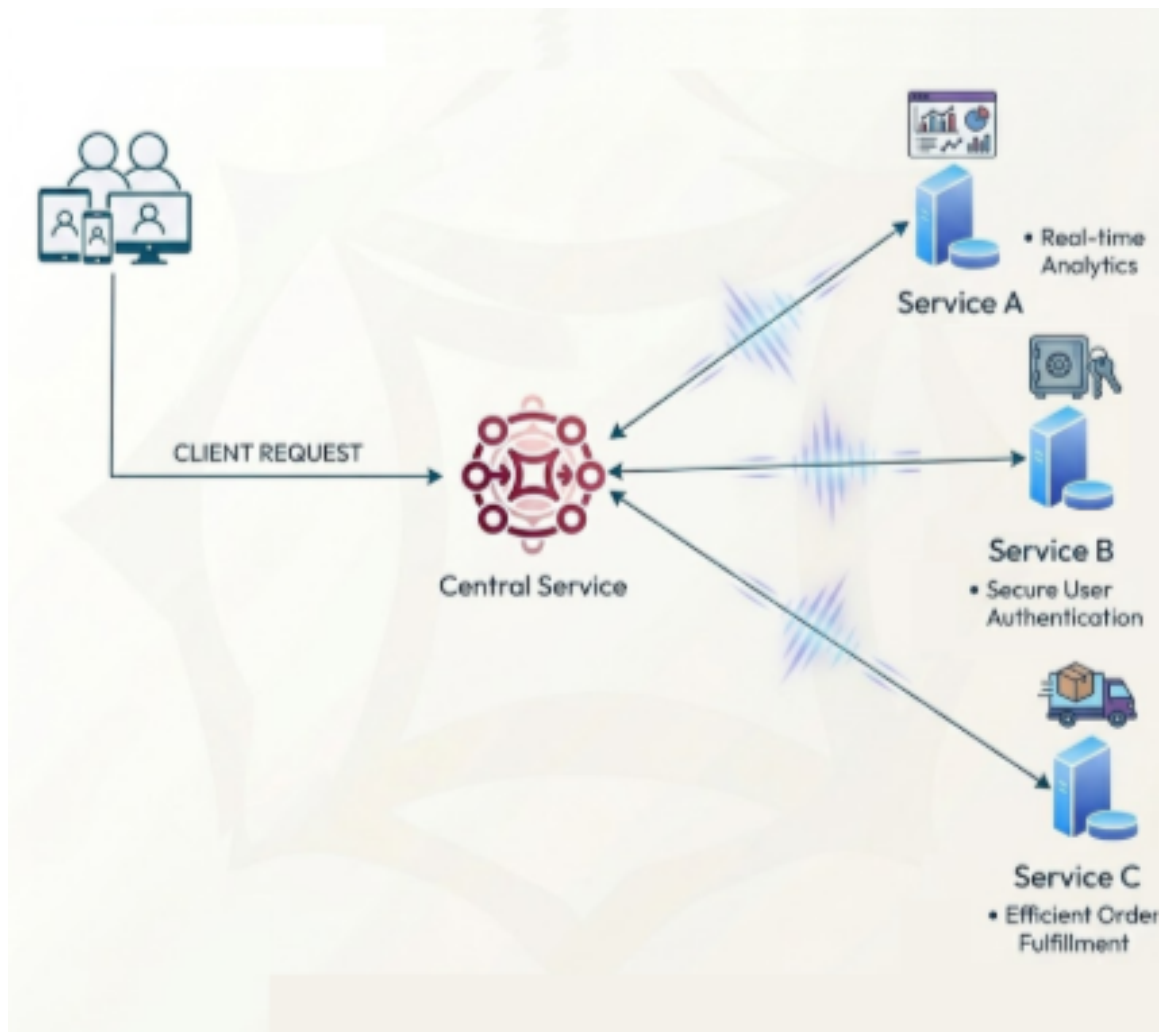


2.6: Saga Pattern

Hình 2.6.

Saga Pattern có hai cách triển khai chính.

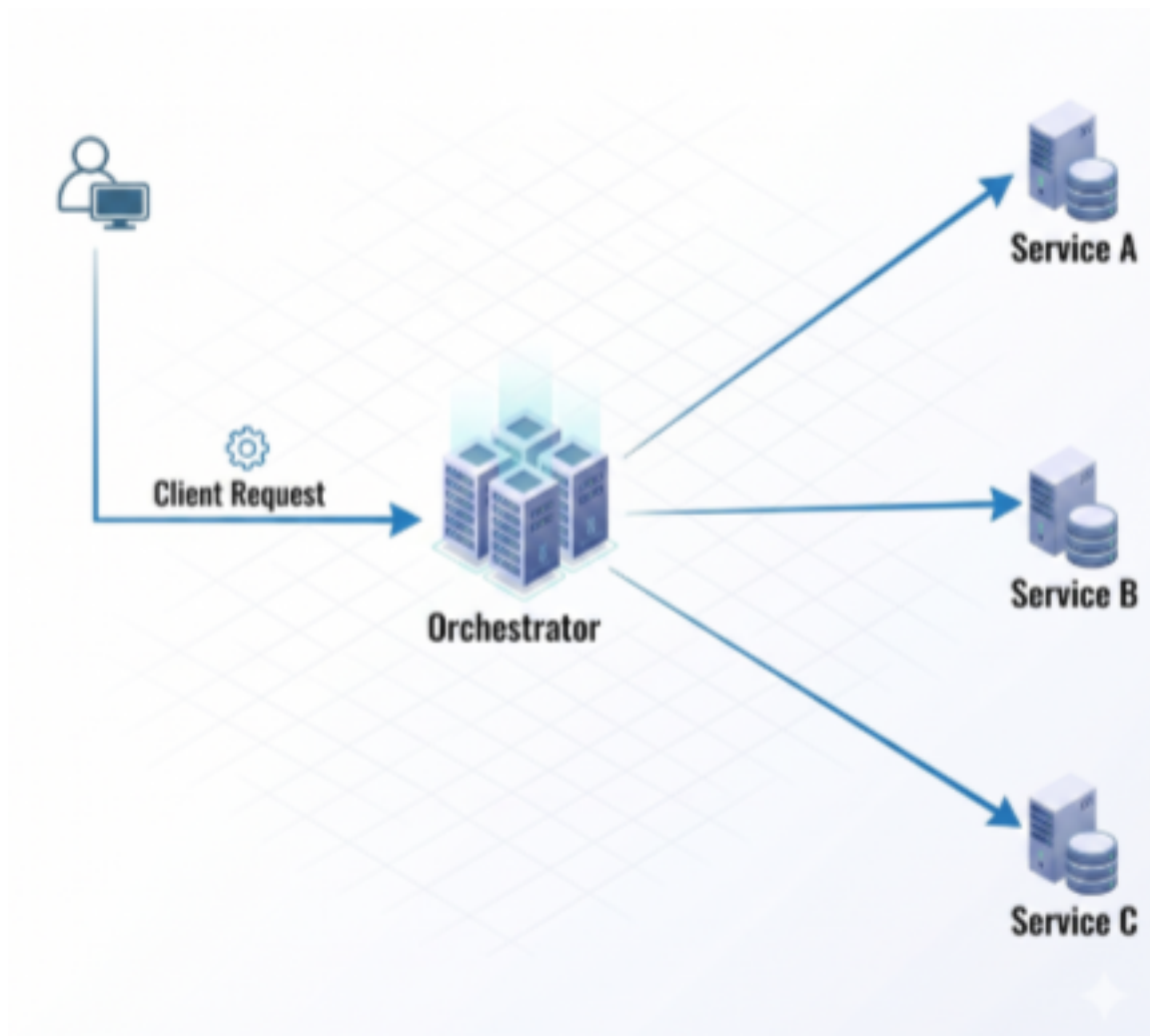
Choreography là mô hình trong đó các dịch vụ giao tiếp với nhau thông qua các sự kiện. Mỗi dịch vụ tự quyết định hành động tiếp theo khi nhận được sự kiện liên quan. Mô hình này có tính mở rộng cao nhưng việc theo dõi và quản lý luồng giao dịch trở nên phức tạp khi số lượng dịch vụ tăng lên.



2.8: Choreography

Hình 2.8.

Orchestration sử dụng một thành phần trung tâm gọi là Orchestrator để điều phối toàn bộ quy trình giao dịch giữa các dịch vụ. Cách tiếp cận này giúp kiểm soát giao dịch tốt hơn và dễ theo dõi luồng xử lý, tuy nhiên làm tăng mức độ phụ thuộc giữa các thành phần trong hệ thống.



2.10: Orchestration

Hình 2.10.

2.7. Khi nào không nên sử dụng Microservices

Mặc dù Microservices mang lại nhiều lợi ích, nhưng không phải mọi hệ thống đều phù hợp với kiến trúc này.

Đối với các dự án nhỏ hoặc có phạm vi nghiệp vụ đơn giản, việc triển khai Microservices có thể làm tăng độ phức tạp không cần thiết. Các nhóm phát triển ít nhân sự cũng thường gặp khó khăn trong việc quản lý và vận hành nhiều dịch vụ độc lập.

Ngoài ra, nếu tổ chức chưa có quy trình DevOps hoàn chỉnh hoặc chưa xây dựng hệ thống giám sát và quản lý tập trung, việc triển khai Microservices có thể dẫn đến nhiều khó khăn trong quá trình vận hành. Trong trường hợp cần phát

triển sản phẩm nhanh để kiểm chứng ý tưởng kinh doanh, kiến trúc Monolithic thường là lựa chọn phù hợp hơn nhờ tính đơn giản và tốc độ triển khai cao.

2.8. Triển khai và cấu hình

Việc triển khai thường dùng container để đảm bảo tính nhất quán môi trường. Docker Compose hỗ trợ khởi tạo đồng thời MySQL và các service, đồng thời tự động import dữ liệu mẫu qua các file SQL.

Chương 3.

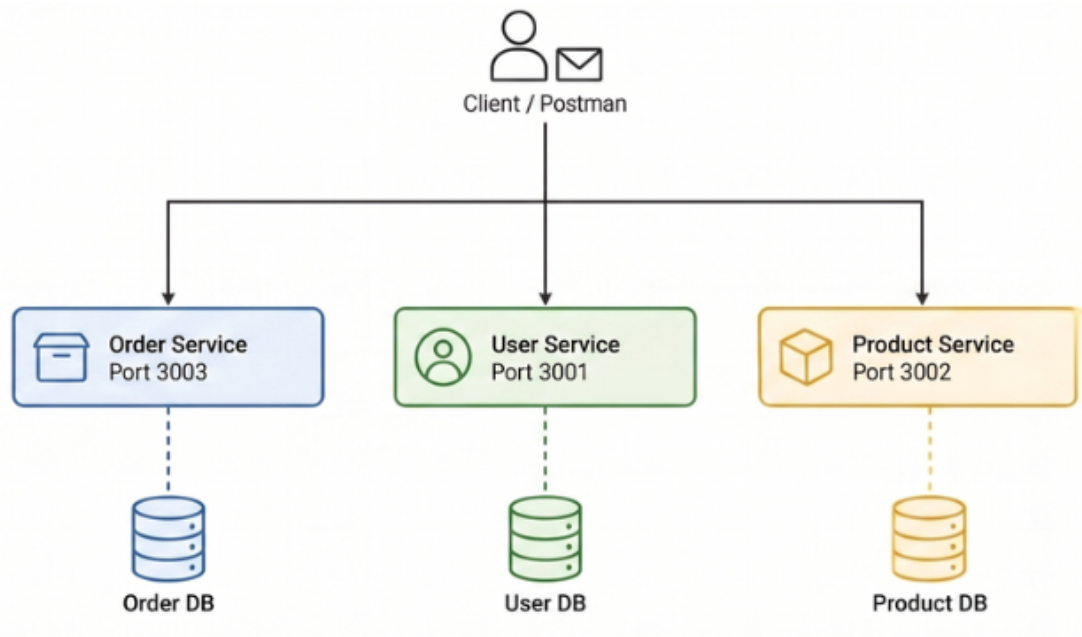
Phân tích và Thiết kế hệ thống

3.1. Yêu cầu chức năng

- User Service: tạo tài khoản người dùng, cập nhật/xóa thông tin người dùng, xem danh sách và thông tin người dùng.
- Product Service: tạo sản phẩm, cập nhật/xóa sản phẩm, xem danh sách và chi tiết sản phẩm.
- Order Service: tạo đơn hàng, cập nhật/xóa đơn hàng, xem danh sách và chi tiết đơn.

3.2. Kiến trúc tổng thể

Hệ thống gồm ba service độc lập chạy trên các port riêng. Mỗi service kết nối tới database riêng trong cùng một máy chủ MySQL. Order Service gọi User Service thông qua HTTP để lấy thông tin người dùng cho mỗi đơn hàng.

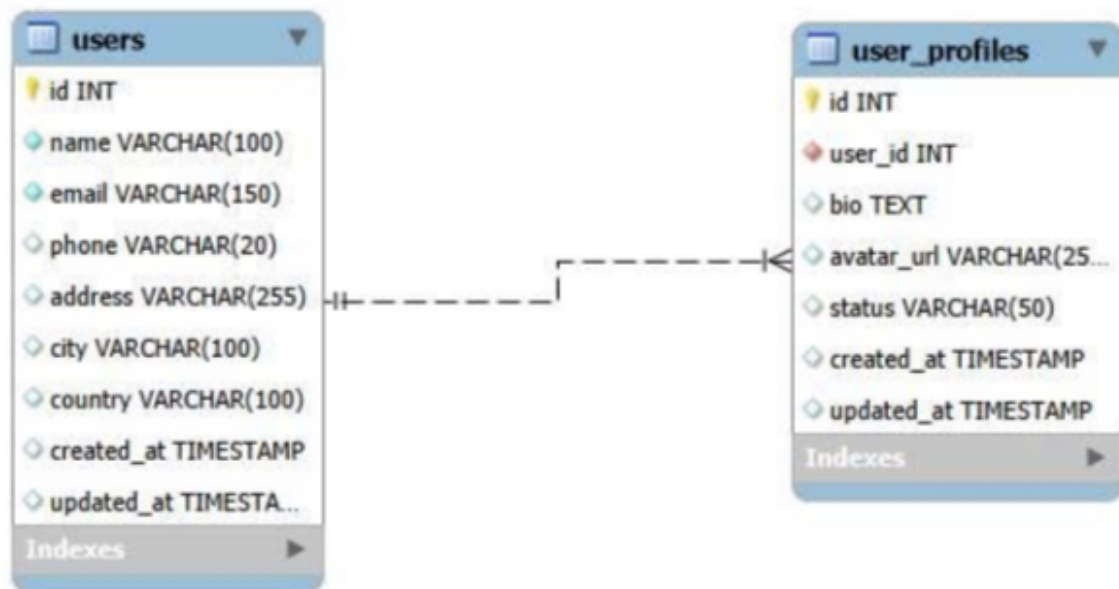


3.2: Kiến trúc hệ thống

Hình 3.2.

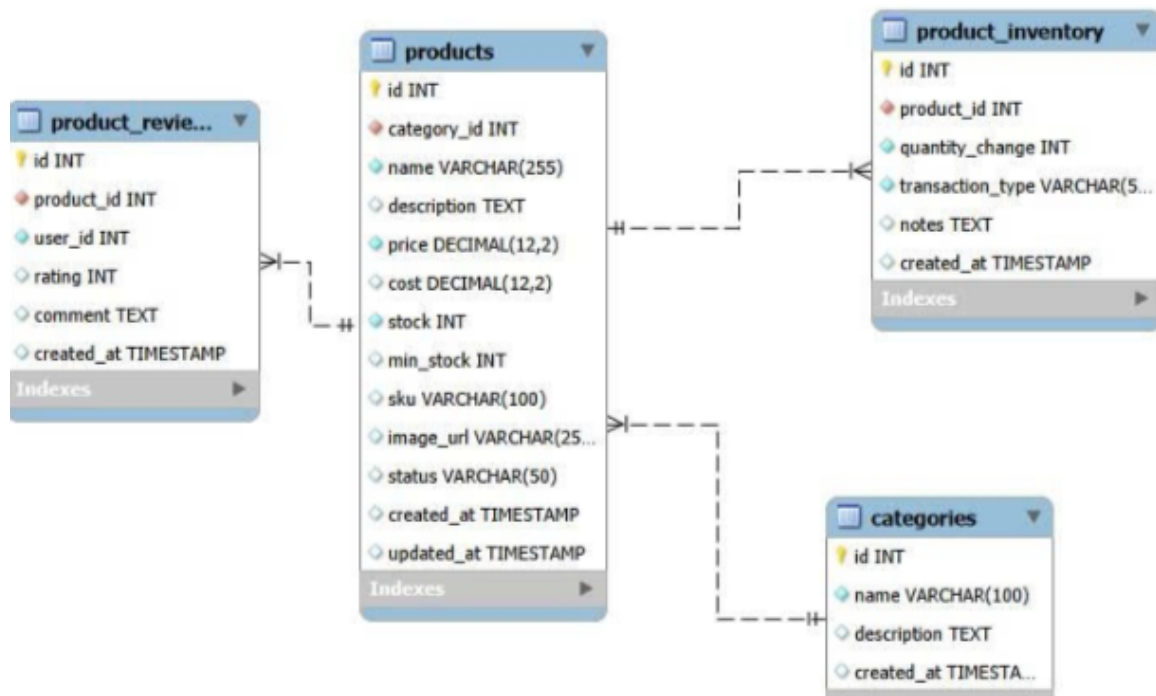
3.3. Thiết kế cơ sở dữ liệu

- User DB: bảng ‘users’, ‘user_profiles’.
- Product DB: bảng ‘categories’, ‘products’, ‘product_reviews’, ‘product_inventory’.
- Order DB: bảng ‘orders’, ‘order_items’, ‘payments’.



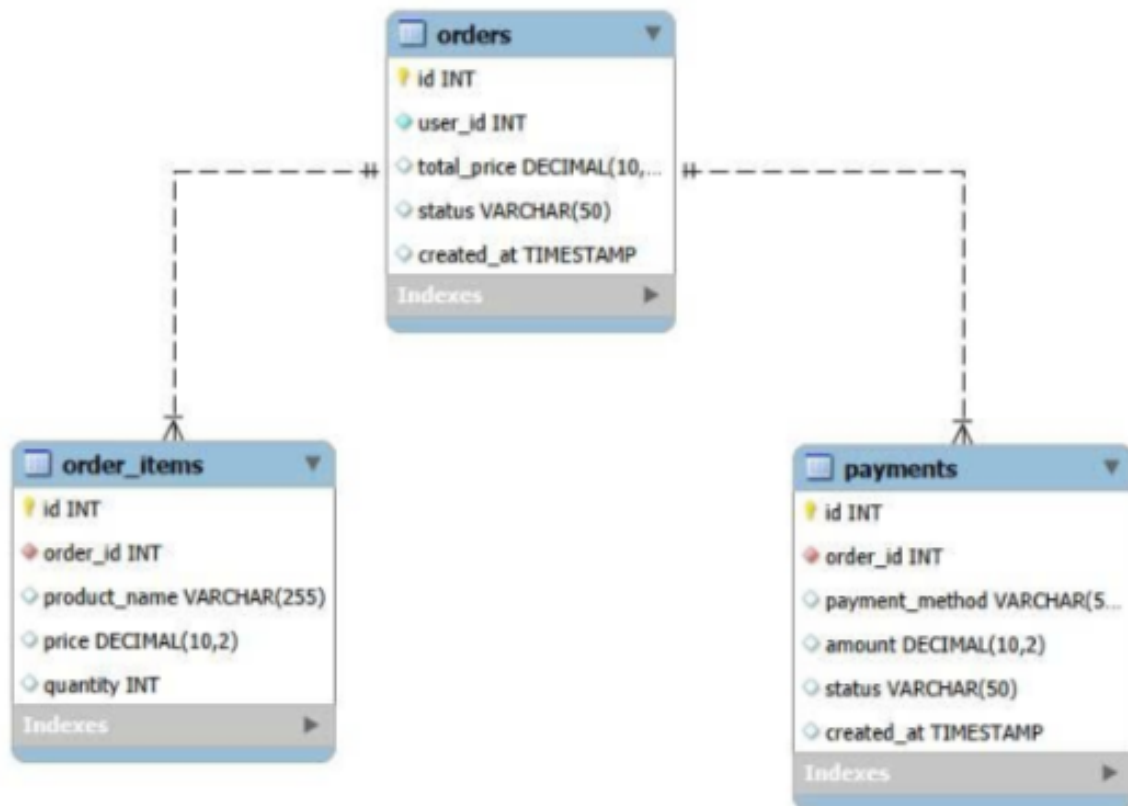
3.4: Sơ đồ EER của user service

Hình 3.4.



3.6: Sơ đồ EER của product service

Hình 3.6.



3.8: Sơ đồ EER của order service

Hình 3.8.

Thiết kế này giúp dữ liệu của từng nghiệp vụ được tách biệt, không có foreign key giữa các database khác nhau.

3.4. Thiết kế API

- User Service: ‘GET /users’, ‘GET /users/:id’, ‘POST /users’, ‘PUT /users/:id’, ‘DELETE /users/:id’.
- Product Service: ‘GET /products’, ‘GET /products/:id’, ‘POST /products’, ‘PUT /products/:id’, ‘DELETE /products/:id’.
- Order Service: ‘GET /orders’, ‘GET /orders/:id’, ‘POST /orders’, ‘PUT /orders/:id’, ‘DELETE /orders/:id’.

3.5. Luồng nghiệp vụ chính

- Tạo đơn hàng: nhận danh sách items, tính tổng tiền, tạo bản ghi `orders`, sau đó thêm các `order_items`.
- Lấy danh sách đơn: truy vấn ‘orders’ và gọi User Service để ghép thông tin người dùng.
- Cập nhật đơn: dùng transaction để cập nhật ‘orders’ và làm mới ‘order_items’ nhằm đảm bảo tính nhất quán.

Chương 4.

Thực nghiệm và Kết quả

4.1. Môi trường thực nghiệm

Hệ thống triển khai bằng NodeJS/Express và MySQL. Các service chạy trên 3 cổng riêng, dữ liệu khởi tạo tự động từ các file SQL. Kiểm thử API thực hiện qua Postman.

4.2. Dữ liệu mẫu

CSDL đã có dữ liệu mẫu gồm:

- 6 người dùng, 6 profile.
- 5 danh mục, 8 sản phẩm, 5 đánh giá.
- 3 đơn hàng, 5 chi tiết đơn, 3 thanh toán.

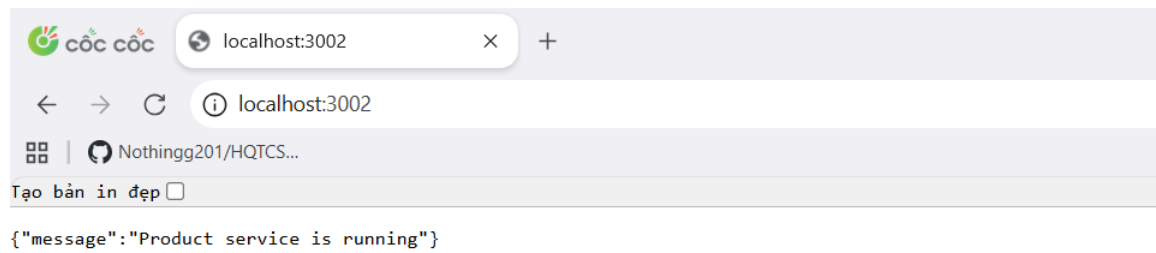
4.3. Kịch bản kiểm thử

4.3.1. Check từng service



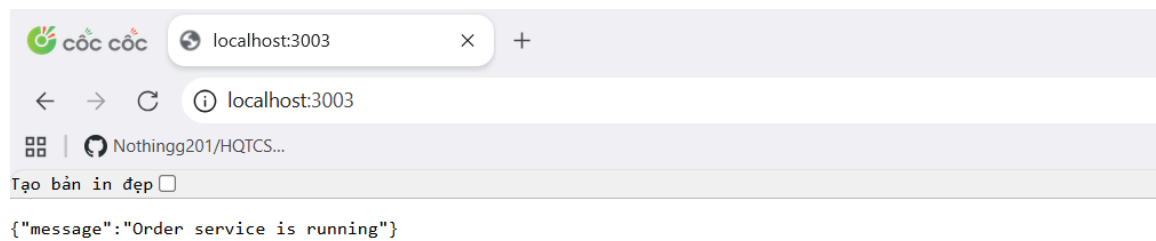
4.2: Check user service

Hình 4.2.



4.4: Check product service

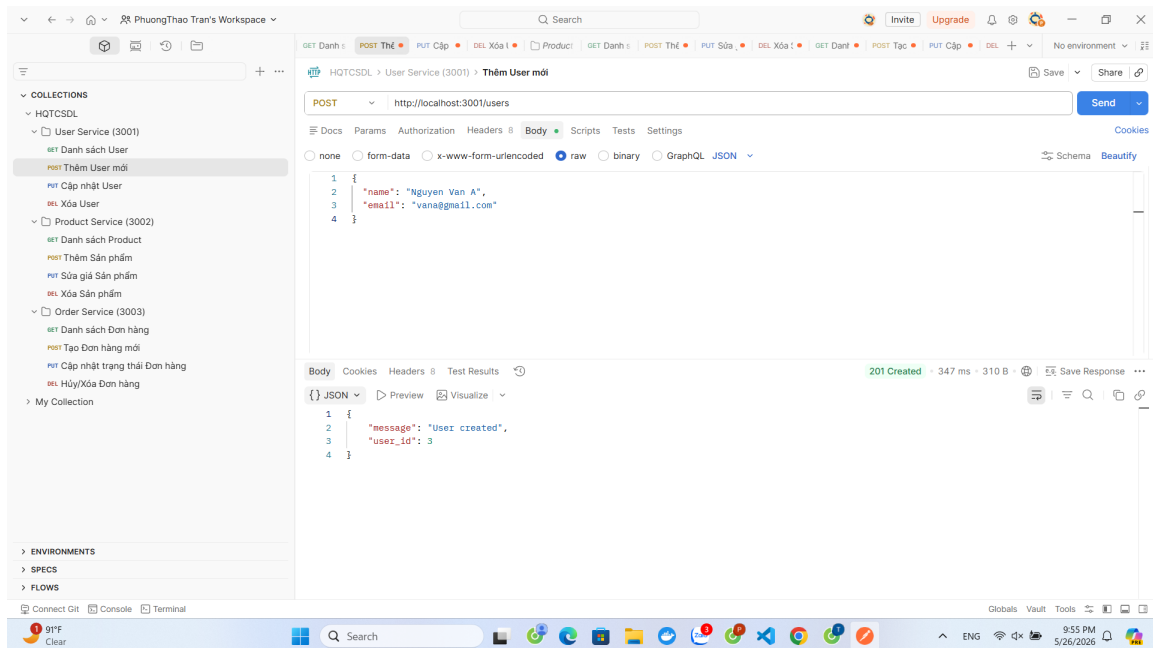
Hình 4.4.



4.6: Check order service

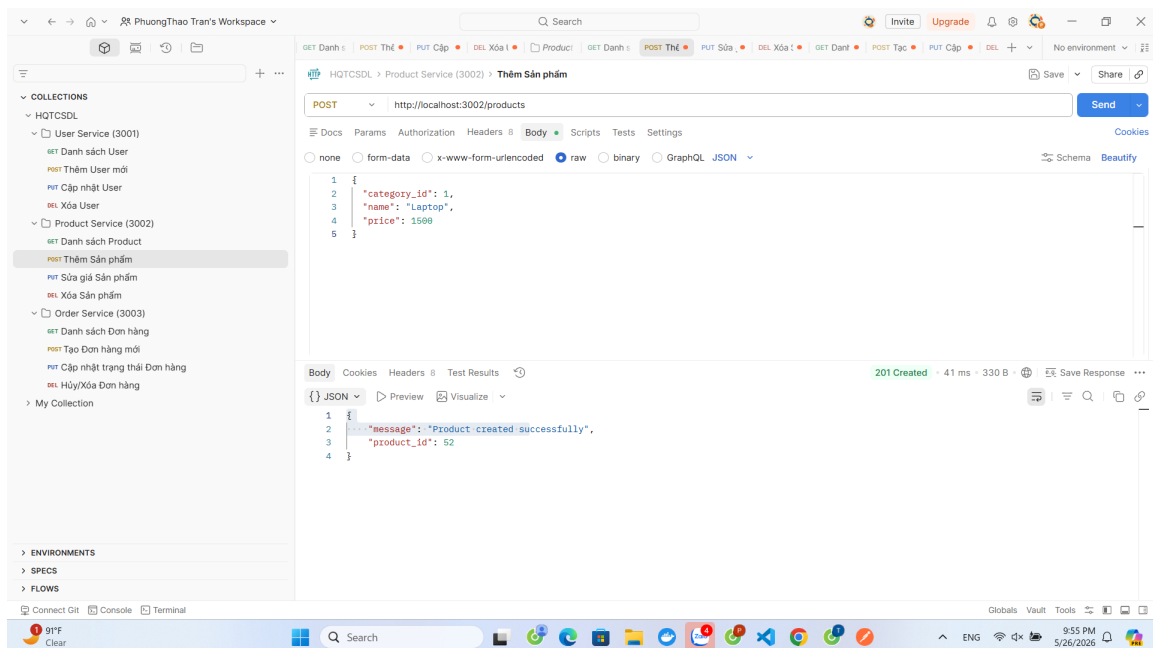
Hình 4.6.

4.3.2. Tạo người dùng và sản phẩm



4.8: Tạo người dùng

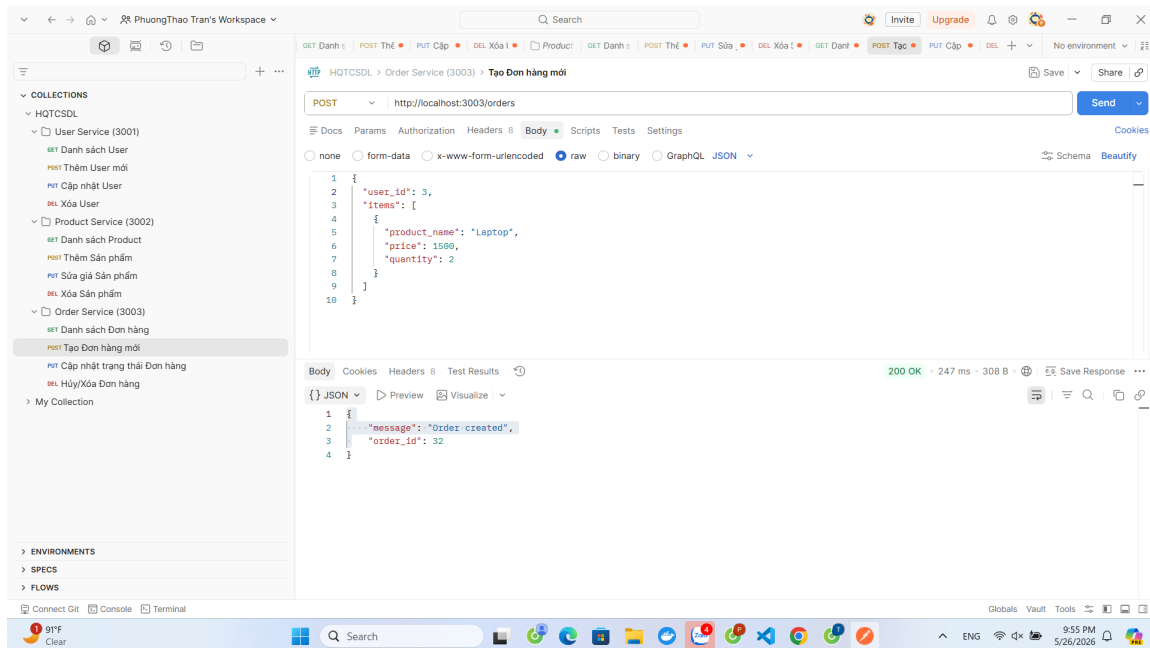
Hình 4.8.



4.10: Tạo sản phẩm

Hình 4.10.

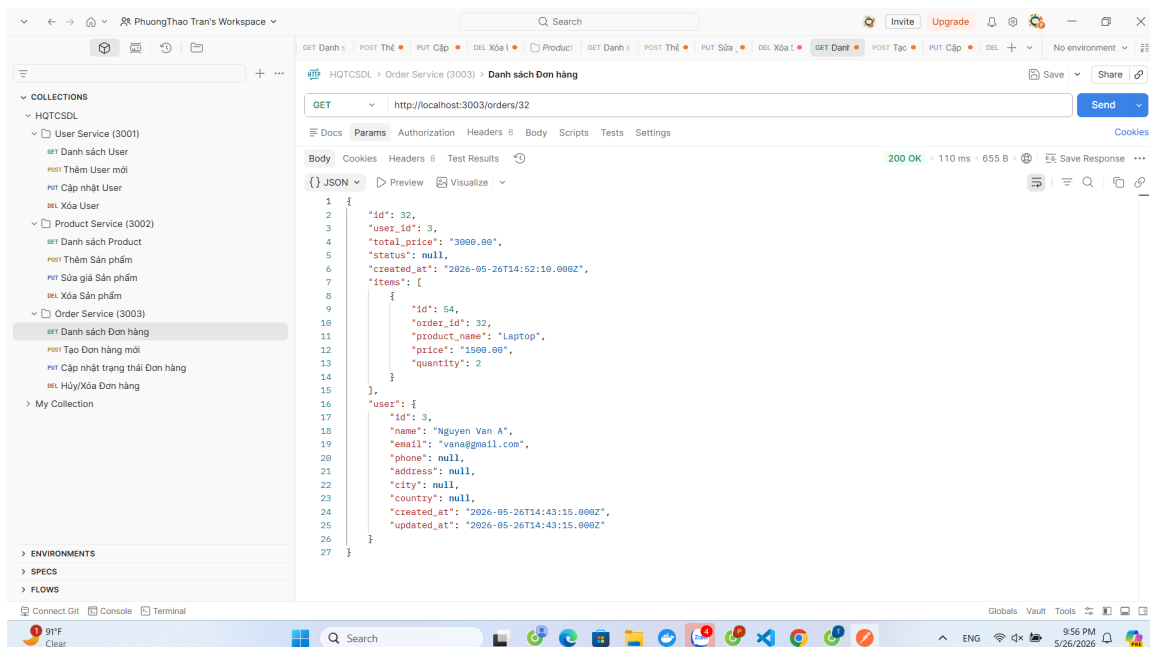
4.3.3. Tạo đơn hàng với nhiều sản phẩm



4.12: Tạo đơn hàng với nhiều sản phẩm

Hình 4.12.

4.3.4. Lấy danh sách đơn hàng



4.14: Lấy danh sách đơn hàng

Hình 4.14.

Đơn hàng đã được ghép thông tin người dùng và tính đúng tổng số tiền.

Stt	Service	Method	Endpoint	Hành động	Body	Status Code	Response time
1	User Service	GET	/users	Lấy danh sách	(Trống)	200 OK	68ms
2	User Service	POST	/users	Thêm User	{ "name": "Nguyen Van Test", "email": "test@gmail.com" }	201 Created	189ms
3	User Service	PUT	/users/12	Cập nhật User	{ "name": "Nguyen Van Test", "email": "test1@gmail.com" }	200 OK	92ms
4	User Service	DELETE	/users/12	Xóa user	(Trống)	200 OK	51ms
5	Product Service	GET	/products	Lấy danh sách	(Trống)	200 OK	93ms
6	Product Service	POST	/products	Thêm Sản phẩm	{ "category_id": 1, "name": "Laptop MSI Gaming", "description": "RTX 4060, 16GB RAM", "price": 32000000, "stock": 5 }	201 Created	66ms
7	Product Service	PUT	/products/13	Sửa giá sản phẩm	{ "name": "Laptop MSI Gaming", "description": "RTX 4060, 16GB RAM", "price": 39000000, "stock": 5 }	200 OK	45ms
8	Product Service	DELETE	/products/13	Xóa sản phẩm	(Trống)	200 OK	42ms
9	Order Service	GET	/orders	Lấy danh sách đơn	(Trống)	200 OK	409ms
10	Order Service	POST	/orders	Tạo Đơn hàng mới	{ "category_id": 1, "name": "Laptop MSI Gaming", "description": "RTX 4060, 16GB RAM", "price": 32000000, "stock": 5 }	200 OK	85ms
11	Order Service	PUT	/orders/8	Cập nhật trạng thái	{ "name": "Laptop MSI Gaming", "description": "RTX 4060, 16GB RAM", "price": 39000000, "stock": 5 }	200 OK	44ms
12	Order Service	DELETE	/orders/8	Hủy/Xóa đơn hàng	(Trống)	200 OK	36ms

4.16: Kết quả 12 test case CRUD cho User Service, Product Service và Order Service

Hình 4.16.

4.4. Kết quả đạt được

- Hệ thống triển khai thành công mô hình 3 service độc lập, tách database theo service.
- Các API CRUD hoạt động theo đúng thiết kế.
- Order Service tổng hợp dữ liệu người dùng từ User Service, đảm bảo không truy cập trực tiếp database của service khác.
- Transaction được dùng để đảm bảo tính nhất quán khi cập nhật/xóa đơn hàng.

Chương 5.

Kết luận và Hướng phát triển

5.1. Kết luận

Bài tập đã xây dựng được hệ thống microservice mẫu với ba service và cơ sở dữ liệu tách biệt. Qua đó làm rõ cách tổ chức dữ liệu theo service và cách các service phối hợp thông qua API.

5.2. Hạn chế

- Chưa có cơ chế đồng bộ dữ liệu nâng cao (Saga, Event-driven).
- Chưa tích hợp xác thực/ủy quyền.
- Chưa có API gateway và hệ thống logging/tracing tập trung.

5.3. Hướng phát triển

- Tích hợp kiểm tra tồn kho hoặc đồng bộ sản phẩm khi tạo đơn.
- Áp dụng message broker để giảm phụ thuộc đồng bộ.
- Bổ sung bảo mật (JWT), giám sát và kiểm thử tự động.